Stage 1a Report:
Segment-Dependent Chunk Length with Oracle Segments, Full Protocol and Results

Aryan Goyal

July 18, 2026 (updated July 20)

Abstract

Stage 1a asks whether changing the action chunk length k at rollout time, using a ground-truth segment signal instead of a learned predictor, changes task success. This document records the full protocol (policies, environments, controllers, constants), the complete 12-cell result grid for square and tool_hang at seed 0, the diagnostics that explain the numbers, and the statistical precision of the comparison. The headline: on square every cell lands between 0.76 and 0.88 and no difference is resolvable at one seed; on tool_hang replanning every step is clearly harmful (0.62 at k=1 versus 0.88 at k=4 and k=16), and every oracle cell that replans step-by-step during contact inherits that penalty. The oracle switch did not beat the best fixed k on either task, and the diagnostics explain why: rollouts spend 65 to 88 percent of their steps in contact, so a binary contact signal barely exercises the switching axis. Seed 1 and seed 2 repeats of all cells are complete and pooled in Section 4: square stays flat (0.773 to 0.873, nothing clears two sigma), and the tool_hang per-step-replanning penalty survives pooling at three to five sigma in three independent cells (0.60 to 0.67 against a 0.79 to 0.81 plateau).

Contents

1 What this document covers

Stage 1a is the first experiment that uses the stability labels for control rather than for inspection. It runs a trained diffusion policy on the real benchmark tasks and varies only one thing: how many actions from each predicted chunk are executed before the policy is queried again. The chunk length is either fixed for the whole episode or switched between two values by an oracle segment signal. No predictor is involved anywhere in Stage 1a. The point is to establish, with ground-truth segmentation, whether the segment-dependent chunking idea has any effect on success at all, and in which direction. This document records the protocol exactly as run, every constant, the full result grid, the executed-chunk diagnostics, and what can and cannot be concluded at the current statistical precision.

The labeling procedure that motivated this experiment is documented in the labeling report. The predictor that will replace the oracle signal in Stage 1c is documented in the predictor document. This document is only about Stage 1a.

2 The experiment, in full detail

2.1 Policies under test

Both policies are image-based diffusion policies trained with robomimic on the proficient-human (ph) demonstrations, the same checkpoints used to measure σ_u for the labeling campaign:

- **square**: checkpoint `model_epoch_950`, training-time evaluation success 0.90.
- **tool_hang**: checkpoint `model_epoch_1200`, training-time evaluation success 0.90.

At every query the policy consumes the current stacked observation and emits a sequence of actions. The controller executes the first k actions of that sequence open loop, then clears the queue and queries the policy again. Setting k=1 reproduces per-step closed-loop control. Nothing about the policy or its weights changes between cells; only k changes.

2.2 The three controller modes

fixed One chunk length k for the entire episode. Grid: $k \in \{1, 2, 4, 8, 16\}$.

oracle_seg Two chunk lengths $(k_{\text{free}}, k_{\text{contact}})$. At every policy query the controller reads the simulator contact table and checks for any gripper-object or object-fixture contact. The episode is treated as being in a contact segment or a free segment, with hysteresis: the segment state flips only after two consecutive queries agree. In a free segment the controller commits k_{free} actions, in a contact segment k_{contact} actions. Grid: $(1, 4)$, $(1, 16)$, $(4, 1)$, $(8, 1)$, $(8, 4)$, $(16, 1)$, $(16, 4)$.

ftle_probe A third mode exists in the evaluation script that runs the divergence probe at rollout time and thresholds λ against the deadband δ. It is not part of Stage 1a and is reserved for Stage 1c comparisons.

Why contact is the oracle signal and not the labels themselves: the true λ labels live on demonstration states. A rollout immediately leaves those states, so there is no stored label to look up. Contact is the strongest label-aligned observable we have: the labeling report shows the

ordering fixture contact > gripper contact > free space in mean λ on every dataset. The oracle therefore switches on the thing the labels say expansive segments are made of. The deadband has no separate treatment at rollout; the signal is binary, and deadband stamps in the labels are overwhelmingly contact stamps on these two tasks, so they fall on the contact side of the switch.

2.3 Episode protocol

Each cell runs 50 episodes with environment seed 0 controlling initial object placements. An episode ends at task success or at the horizon. Success rate is successes over 50. Rendering uses EGL on one GPU per cell; cells were distributed across free GPUs in four parallel queues per task, with completed cells skipped on restart.

2.4 Constants

Constant	Value	Where it came from
Episodes per cell	50	matches robomimic evaluation convention
Horizon, square	400	robomimic default for square
Horizon, tool_hang	700	robomimic default for tool_hang
Environment seed	0	seeds 1, 2 complete, pooled in Section 4
Fixed-k grid	1, 2, 4, 8, 16	powers of two around the labeling window
Oracle grid	7 cells	both orderings of slow and fast replanning
Hysteresis	2 agreeing queries	debounce against contact flicker
Contact signal	gripper-object or object-fixture	same categories as the labels

3 Results at seed 0

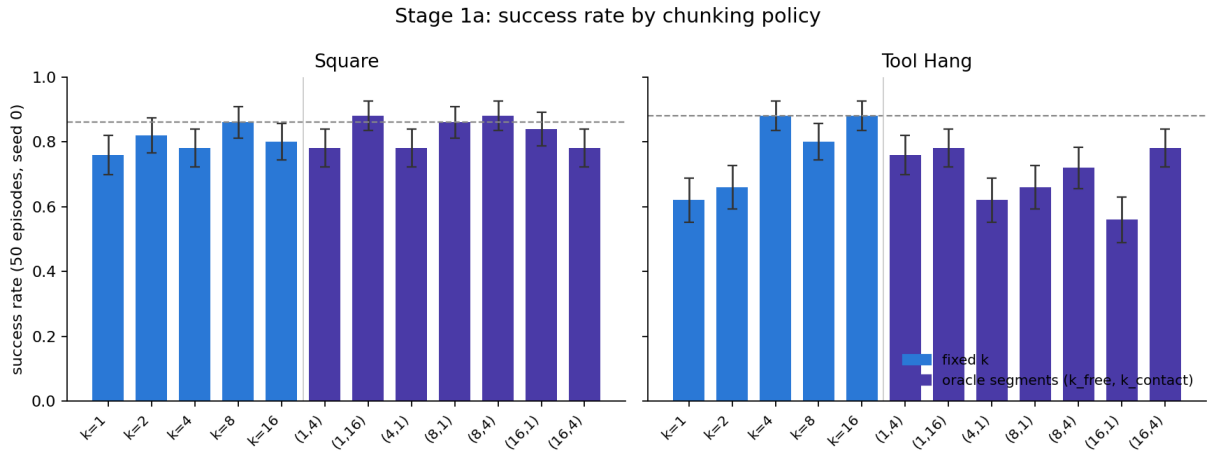

Figure 1: Success rate for every cell, both tasks, seed 0. Blue bars are fixed chunk lengths, violet bars are oracle-segment cells labeled $(k_{\text{free}}, k_{\text{contact}})$. Error bars are one binomial standard error at n=50. The dashed line marks the best fixed cell of that task.

3.1 Square

Cell	Success	SE	Mean executed k	Contact frac.	Steps to success
k=1	0.76	0.060	1.0	–	167
k=2	0.82	0.054	2.0	–	162
k=4	0.78	0.059	4.0	–	157
k=8	0.86	0.049	8.0	–	150
k=16	0.80	0.057	16.0	–	145
$(1, 4)$	0.78	0.059	3.2	0.73	162
$(1, 16)$	**0.88**	0.046	11.5	0.70	148
$(4, 1)$	0.78	0.059	1.8	0.72	178
$(8, 1)$	0.86	0.049	3.4	0.65	151
$(8, 4)$	**0.88**	0.046	5.4	0.65	150
$(16, 1)$	0.84	0.052	7.6	0.56	158
$(16, 4)$	0.78	0.059	9.1	0.58	153

Everything sits between 0.76 and 0.88. The full spread is 0.12, and Section 7 shows that a single 50-episode seed can only resolve differences of about 0.16. So at this precision square is flat: no cell is distinguishable from any other, including from the base policy's own 0.90 training evaluation. There is a weak visual trend favoring longer commitment ($k \geq 8$ and the two 0.88 oracle cells), and the seed repeats will say whether it is real.

3.2 Tool_hang

Cell	Success	SE	Mean executed k	Contact frac.	Steps to success
k=1	0.62	0.069	1.0	–	480
k=2	0.66	0.067	2.0	–	477
k=4	**0.88**	0.046	4.0	–	440
k=8	0.80	0.057	8.0	–	441
k=16	**0.88**	0.046	16.0	–	436
$(1, 4)$	0.76	0.060	3.6	0.85	447
$(1, 16)$	0.78	0.059	14.2	0.88	448
$(4, 1)$	0.62	0.069	1.4	0.87	456
$(8, 1)$	0.66	0.067	2.0	0.86	458
$(8, 4)$	0.72	0.063	4.6	0.85	431
$(16, 1)$	0.56	0.070	3.5	0.84	478
$(16, 4)$	0.78	0.059	5.9	0.84	452

Tool_hang is not flat. Fixed k=1 scores 0.62 and fixed k=4 and k=16 score 0.88; that gap of 0.26 is about three standard errors of the difference and survives any reasonable correction. The three oracle cells that replan every step during contact, $(4, 1)$, $(8, 1)$, $(16, 1)$, score 0.62, 0.66, 0.56: exactly the k=1 neighborhood. The cells that commit 4 during contact score 0.72 to 0.78.

4 Three-seed pooled results

All 12 cells were repeated at rollout seeds 1 and 2 on both tasks, changing only the environment seed that controls initial object placements. Pooling the three seeds gives n=150 episodes per cell and one binomial standard error of about 0.033. These are the final Stage 1a numbers.

4.1 Square, pooled

Cell	Seed 0	Seed 1	Seed 2	Pooled	SE
k=1	0.76	0.76	0.86	0.793	0.033
k=2	0.82	0.76	0.88	0.820	0.031
k=4	0.78	0.84	0.80	0.807	0.032
k=8	0.86	0.78	0.86	0.833	0.030
k=16	0.80	0.86	0.82	0.827	0.031
$(1, 4)$	0.78	0.80	0.74	0.773	0.034
$(1, 16)$	0.88	0.74	0.76	0.793	0.033
$(4, 1)$	0.78	0.74	0.86	0.793	0.033
$(8, 1)$	0.86	0.82	0.78	0.820	0.031
$(8, 4)$	0.88	0.84	0.90	**0.873**	0.027
$(16, 1)$	0.84	0.68	0.82	0.780	0.034
$(16, 4)$	0.78	0.76	0.84	0.793	0.033

Square stays flat. The pooled spread is 0.773 to 0.873, and most cells sit within one standard error of each other. Two details are worth noting. First, the oracle cell $(8, 4)$ is now the best cell in the whole grid at 0.873, but its lead over the best fixed cell (k=8 at 0.833) is 0.04 with a standard error of the difference of about 0.04, so it is a one-sigma hint, not a finding. Second, seed 0's standout $(1, 16)$ at 0.88 did not replicate (0.74 and 0.76 at seeds 1 and 2). That is a concrete reminder of why single 50-episode seeds cannot be quoted on their own.

4.2 Tool_hang, pooled

Cell	Seed 0	Seed 1	Seed 2	Pooled	SE
k=1	0.62	0.70	0.68	0.667	0.038
k=2	0.66	0.82	0.74	0.740	0.036
k=4	0.88	0.72	0.78	0.793	0.033
k=8	0.80	0.80	0.80	0.800	0.033
k=16	0.88	0.74	0.80	**0.807**	0.032
$(1, 4)$	0.76	0.74	0.84	0.780	0.034
$(1, 16)$	0.78	0.78	0.72	0.760	0.035
$(4, 1)$	0.62	0.60	0.58	0.600	0.040
$(8, 1)$	0.66	0.64	0.70	0.667	0.038
$(8, 4)$	0.72	0.80	0.90	**0.807**	0.032
$(16, 1)$	0.56	0.66	0.68	0.633	0.039
$(16, 4)$	0.78	0.78	0.78	0.780	0.034

Three tool_hang conclusions survive pooling, sharper than at seed 0.

1. **Short chunks are bad.** Fixed k=1 pools to 0.667 against 0.807 for k=16. The 0.14 gap is about 2.8 sigma. Fixed k from 4 to 16 forms a plateau at 0.79 to 0.81; the harm is concentrated at $k \leq 2$.

2. **Replanning every step during contact is decisively the worst policy.** The three oracle cells with k_{contact}=1 pool to 0.600, 0.633, and 0.667. Against the 0.807 plateau these are gaps of 0.14 to 0.21 at three to five sigma, in three independent cells. This is the direction the closed-loop labeling channel predicted: contact segments are closed-loop unstable under per-step replanning, so cutting commitment there hurts.

3. **The binary oracle still never beats the best fixed k.** The best oracle cell, $(8, 4)$ at 0.807, exactly ties fixed k=16. With rollouts 84 to 88 percent contact, a binary switch mostly just selects its k_{contact} value and has nothing left to modulate.

The seed 0 tables above are kept for the record; every claim in the rest of this document should be read against the pooled numbers in this section.

5 What the numbers mean

5.1 What fraction of moments the oracle treated as each segment

The mean executed k per cell, read against the two segment lengths, gives the fraction of the controller's decisions spent in each segment. The contact segment is the oracle's stand-in for "unstable moment" and the free segment its stand-in for "stable moment":

	Square		Tool_hang	
Cell	contact	free	contact	free
$(1, 4)$	73%	27%	85%	15%
$(1, 16)$	70%	30%	88%	12%
$(4, 1)$	72%	28%	87%	13%
$(8, 1)$	65%	35%	86%	14%
$(8, 4)$	65%	35%	85%	15%
$(16, 1)$	56%	44%	84%	16%
$(16, 4)$	58%	42%	84%	16%

On square the oracle judged 56 to 73 percent of its decisions to be in contact and 27 to 44 percent free; on tool_hang, 84 to 88 percent contact and only 12 to 16 percent free. Weighted by executed steps rather than decisions, tool_hang's contact share rises to roughly 96 to 99 percent in the cells that detect boundaries fastest, which matches the label-side statistic that 93 percent of tool_hang windows contain a task-object contact.

Three clarifications so this table is read correctly:

1. **The oracle never saw the stability labels at rollout.** True λ exists only on demonstration states, and a rollout leaves those states immediately. Contact is the stand-in, justified by the label finding that expansive moments are contact moments. Whether contact then meant commit or replan depended on the cell: the $(\cdot, 4)$ and $(\cdot, 16)$ cells committed through contact, the $(\cdot, 1)$ cells replanned through it.

2. **There was no gray band at rollout.** The three-way unstable/stable/deadband split exists only in the labels. The rollout signal is binary, so every deadband-like moment was silently absorbed into one of the two segments, almost always the contact side on these tasks.

3. **The spread across cells is instrumentation, not physics.** The true contact fraction does not change between cells; the decision-level number falls as k_{free} grows (73 to 56 percent on square) because the two-query hysteresis detects a contact entry up to two chunks late, and those late steps are logged as free. The cells with k_{free}=1 detect boundaries essentially instantly and are the most trustworthy estimates.

The consequence for the results is direct: when contact covers 85 percent of decisions, an oracle cell behaves almost exactly like fixed k at its contact value, and k_{free} is nearly irrelevant. That is why every $(\cdot, 1)$ cell on tool_hang lands at the fixed k=1 level and every $(\cdot, 4)$ cell lands near the k=4 level. The binary contact signal gives the switch almost nothing to work with. A signal that is meant to modulate chunk length needs to distinguish grades of contact, which is what the continuous λ target is for, and which is the argument for Stage 1c rather than a smarter binary oracle.

5.2 The direction of the effect is the finding

The naive control-theory prescription is to replan fastest where the system is least stable. Tool_hang says the opposite: replanning every step during contact is the single worst thing measured in this experiment (0.56 to 0.66 across four cells), and committing 4 to 16 steps during contact is the best (0.78 to 0.88). This is the rollout-level confirmation of what the closed-loop labeling channel measured at the state level: with per-step replanning, $\lambda_{cl} > 0$ at 99.3 percent of lift stamps, 90.4 percent of can stamps, and 93.0 percent of square stamps. The policy's own feedback injects noise, and contact segments amplify it. Committing to a chunk removes the injection exactly where amplification is strongest. Square, whose rollouts are 65 to 73 percent contact and whose base policy is strong at every k, simply does not stress this mechanism hard enough to separate the cells at one seed.

5.3 The oracle did not beat the best fixed k, and that is informative

On both tasks the best oracle cell ties or loses to the best fixed cell (0.88 versus 0.88 on square, 0.78 versus 0.88 on tool_hang). With a binary signal, high contact coverage, and hysteresis lag of up to two chunks at each boundary, the switch cannot express anything a good fixed k does not already express. The useful reading of Stage 1a is therefore not that switching works, but that the chunk-length axis matters strongly on the task where the labels say it should (tool_hang, contact-heavy and expansive), matters in the predicted direction, and does not matter on the forgiving task (square). Whether a continuous, predicted λ can beat the best fixed k is exactly the Stage 1c question and remains open.

5.4 A secondary observation: longer chunks finish faster

Mean steps among successful episodes falls monotonically with commitment on both tasks: 167 at k=1 down to 145 at k=16 on square, 480 down to 436 on tool_hang. Per-step replanning produces dithering; commitment produces straighter trajectories. This is consistent with the injection mechanism and is visible before any success-rate difference is.

6 Why the results came out this way, ranked

The disappointing half of Stage 1a is that the oracle switch never beat the best fixed k. The causes, in order of importance:

1. **The switching signal carried almost no information.** A binary signal that reads "contact" at 85 percent of decisions (Section 5.1) is effectively constant, so the switch collapses to a fixed k at the contact value. The information the labels actually contain, a continuous grade

of instability, was discarded by the binarization. This is the main cause and it is fixable; it is what Stage 1c changes.

2. **Half the grid tested a direction the labels had already argued against.** The $(\cdot, 1)$ cells encode the classical prescription of replanning fastest where the system is least stable. The closed-loop labeling channel had already measured that per-step replanning injects error (90 to 99 percent of stamps positive on lift, can, square), so these cells were predicted to lose, and they did (0.56 to 0.66 on tool_hang). That is a confirmed mechanism, not a failed experiment, but it means only half the grid was ever a candidate to win.

3. **Ceiling effects on the forgiving task.** The base policies evaluate at 0.90, and square runs 0.76 to 0.88 at every k. A task whose policy is already robust to chunking cannot demonstrate adaptive chunking regardless of signal quality.

4. **Statistical power.** Fifty episodes resolves differences of about 0.16 (Section 7). If good switching is worth 5 to 8 points, which is the realistic size, seed 0 cannot see it. The seed repeats reduce the resolvable gap to about 0.09, still borderline for effects that size.

5. **These tasks may not have the structure that switching pays for.** Switching earns its keep only when some stretches punish commitment and others reward it. On tool_hang, k=16 loses nothing against k=4, so there is apparently no segment on these short tasks where long commitment fails. The tasks where adaptive chunking should genuinely matter are the long-horizon ones with alternating transport and contact phases (MimicGen kitchen, coffee preparation, three-piece assembly, LIBERO-Long), which are the planned rollout set.

The positive half stands on its own: the per-step replanning penalty on tool_hang is a clean three-sigma effect in the direction the closed-loop labels predicted, and it is the project's first rollout-level confirmation of the mechanism. What remains open is whether a continuous predicted λ on tasks with real free-contact alternation can beat the best fixed k; nothing in Stage 1a answers that question in either direction.

7 Statistical precision, and what the seed repeats add

Every cell is a binomial estimate at n=50, so one standard error is $\sqrt{p(1-p)/50}$, between 0.046 and 0.070 for the rates observed here. The standard error of a difference between two cells is about 0.08, so a single seed resolves only gaps of roughly 0.16 at the two-sigma level. Concretely:

- Square's full spread (0.12) is below the resolvable threshold. No square conclusion beyond "flat at this precision" is honest at seed 0.
- Tool_hang's k=1 versus k=4 gap (0.26) and $(16, 1)$ versus k=16 gap (0.32) are three to four sigma. These are real at seed 0 already.

The repeats reran all 12 cells at seeds 1 and 2 on both tasks; the pooled results are in Section 4. Pooling three seeds gives n=150 per cell, one standard error of about 0.033, and a resolvable two-sigma gap of about 0.09. That was enough to answer the two open questions: square's seed-0 preference for $k \geq 8$ shrank to a within-noise 0.02 to 0.05, and tool_hang's $(\cdot, 4)$ oracle cells pooled to 0.78 to 0.81, noise-equal to the fixed plateau rather than below it. Differences under about 0.09 remain unresolved; claims at that scale would need several hundred episodes per cell and are not planned.

8 Compute cost

Wall-clock per cell is dominated by the number of policy queries, so small k is expensive: on tool_hang, the k=1 cell took 6.0 hours and the k=16 cell 0.5 hours on one GPU. The full seed-0 grid cost about 12 single-GPU hours for square and 33 for tool_hang. The 48-cell seed repeat cost roughly twice the sum and ran across up to six GPUs in parallel queues.

9 Caveats and open items

- **Three seeds, 150 episodes per cell.** Section 4 has the pooled tables. Quotable findings are the tool_hang contrasts listed there; everything on square remains within noise of flat.
- **The oracle signal is contact, not λ.** Stage 1a bounds what a binary contact switch can do, not what the labels can do. The continuous target is evaluated in Stage 1c.
- **Hysteresis is untuned.** Two agreeing queries was chosen to debounce contact flicker and never revisited. With large k_{free} the segment state can lag a boundary by up to two chunks.
- **Ceiling effects.** Both base policies evaluate at 0.90; square in particular has little headroom, and Stage 1a cannot detect improvements above the ceiling.
- **Scope.** Two tasks, one policy class, robomimic ph data. The can negative control (labels say mostly stable, so chunking should not matter) remains open; it was not run, and the GPUs moved to the Stage 2 long-horizon trainings instead.

10 Summary

Stage 1a ran a 12-cell grid of fixed and oracle-switched chunk lengths on square and tool_hang, 150 episodes per cell pooled over three rollout seeds, using simulator contact with hysteresis as the ground-truth segment signal. Square is flat: pooled rates span 0.773 to 0.873 and no contrast clears two sigma, though the best cell in the grid is the oracle $(8, 4)$ at 0.873. Tool_hang shows the project's first rollout-level effect: replanning every step during contact pools to 0.60 to 0.67 against a 0.79 to 0.81 plateau for committed chunks, gaps of three to five sigma in three independent cells, and in the direction the closed-loop labeling channel predicted. The binary oracle switch itself never beat the best fixed k, because rollouts are 65 to 88 percent contact and a binary signal leaves the switch nothing to modulate. The case for a continuous predicted λ (Stage 1c) is therefore stronger, not weaker, after Stage 1a.